LAB 4 - WEEK 5

APACHE AVRO LAB – FOR DOCKER

1. [2] **Avro Word Count**
   Try to run through all the steps and see if they work properly for you.

   Submit the java and output files.

2. [3] **Avro Station-Temperature-Year**

   Submit the java and output files along with the schema file. Also submit the screenshot of the JSON produced by avro-tools for the output file.

3. [5] **Avro Max Temperature**

   Submit all the java files, schema file and the output file along with the screenshot of the JSON produced by avro-tools.

© 2026-28 Mrudula Mukadam, Maharishi International University * all rights reserved

1. Word Count as Avro Data file

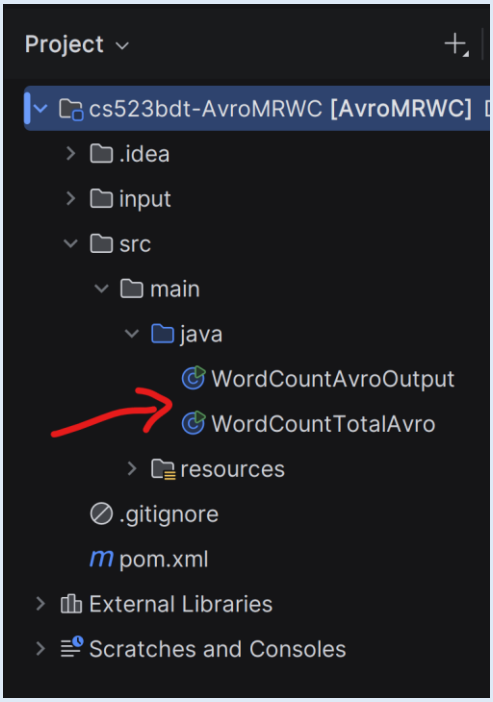
The purpose of this lab is to give you a feel of how Avro works with MapReduce.

Make sure that you can run the given *Avro Word Count* programs both locally and in the pseudo-distributed mode (Docker environment) by following these given steps.

1. Get both the Avro word count programs by cloning the following git repository:

   `git clone https://github.com/himrudula/cs523bdt-AvroMRWC.git`

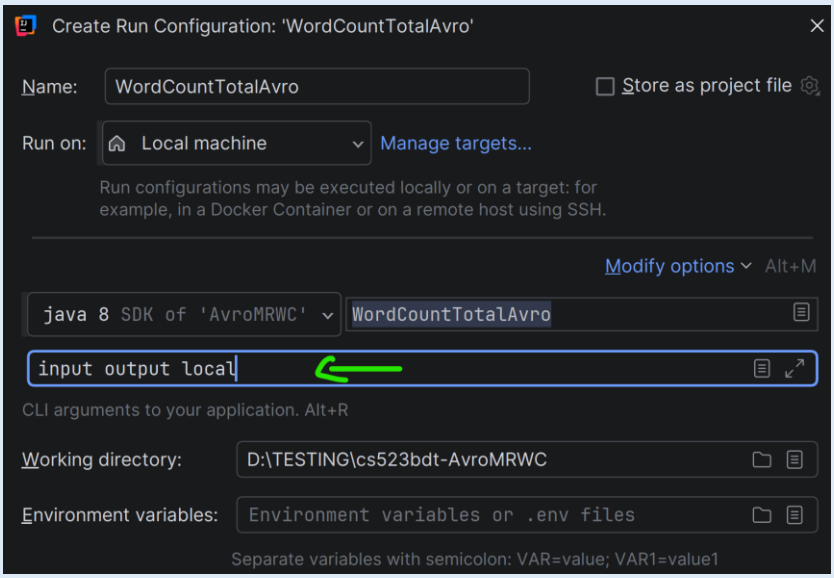
2. Open the project folder "cs523bdt-AvroMRWC" in any IDE like IntelliJ.
3. In the project structure, you should find two java files: *WordCountAvroOptput.java and WordCountTotalAvro.java.*
4. Note that some required lines in the java code are purposefully omitted or commented. You need to find those and correct the given java programs to proceed to the next steps. Take help from the lecture videos.



Local Mode:

1. Do a `mvn clean install` and make sure that there are no compilation errors in your project.
2. You need to run either *WordCountAvroOptput.java* or *WordCountTotalAvro.java* file.
3. Make sure to supply runtime arguments to your program as "input", "output" and "local" as shown below:

© 2026-28 Mrudula Mukadam, Maharishi International University * all rights reserved



4. Run the Java program and see that the Avro output data file is created as *"part-r-00000.avro".* This file has data in binary format and schema is also attached to it.
5. Next, we'll use the Avro tools (which is written in Java and is available in our Docker container) to display the contents of *part-r-00000.avro* file.
6. Make sure that the Hadoop Docker container is running. Start the shell for this container:

   ```
   docker exec -it cs523bdt-lab bash
   ```

7. Make sure that the part-r-00000.avro file is copied to the "my_code" directory of your Docker container.
8. Now from the container shell, run the following command. The ***tojson*** command converts an Avro datafile to JSON and prints it to the console:

   ```
   avro-tools tojson my_code/part-r-00000.avro
   ```

   Or for Pretty JSON, run the following command:

   ```
   avro-tools tojson --pretty my_code/part-r-00000.avro
   ```

Pseudo-distributed Mode:

1. Using `mvn package` command, create a jar of your program. Make sure that the class name is available in the manifest file.
2. Put the jar file inside "my_code" for your Docker container and also make sure that the input file is also available under "my_code".

```
[2026-03-10 12:18] root@localhost /opt $ ls my_code -l
total 36
-rwxrwxrwx 1 root root  9458 Mar 10 10:55 AvroMRWC-1.0.jar
-rwxrwxrwx 1 root root 21769 Mar 10 10:43 Lab2-WC-Input.txt
[2026-03-10 12:18] root@localhost /opt $
```

© 2026-28 Mrudula Mukadam, Maharishi International University * all rights reserved

3. Run the following commands now to run the jar file which will generate the part-r-00000.avro output file.
   - ☞ **hadoop fs -mkdir /user/input**
   - ☞ **hadoop fs -put my_code/Lab2-WC-Input.txt /user/input**
   - ☞ **hadoop jar my_code/AvroMRWC-1.0.jar WordCountAvroOutput /user/input /user/output**

4. Check that the part-r-00000.avro file is produced in the /user/output directory.

```
[2026-03-10 12:20] root@localhost /opt $ hadoop fs -ls /user/output -l
Found 2 items
-rw-r--r--   1 root supergroup          0 2026-03-10 12:20 /user/output/_SUCCESS
-rw-r--r--   1 root supergroup       3005 2026-03-10 12:20 /user/output/part-r-00000.avro
ls: `-l': No such file or directory
[2026-03-10 12:21] root@localhost /opt $ |
```

5. Now transfer this output file from HDFS to the container local file system (my_code).

   hadoop fs -get /user/output/*.avro my_code

6. Now from the container shell, run the following command. The ***tojson*** command converts an Avro datafile to JSON and prints it to the console:

   avro-tools tojson my_code/part-r-00000.avro

   Or for Pretty JSON, run the following command:

   avro-tools tojson --pretty my_code/part-r-00000.avro

2. Station-Temperature-Year as Avro Data File

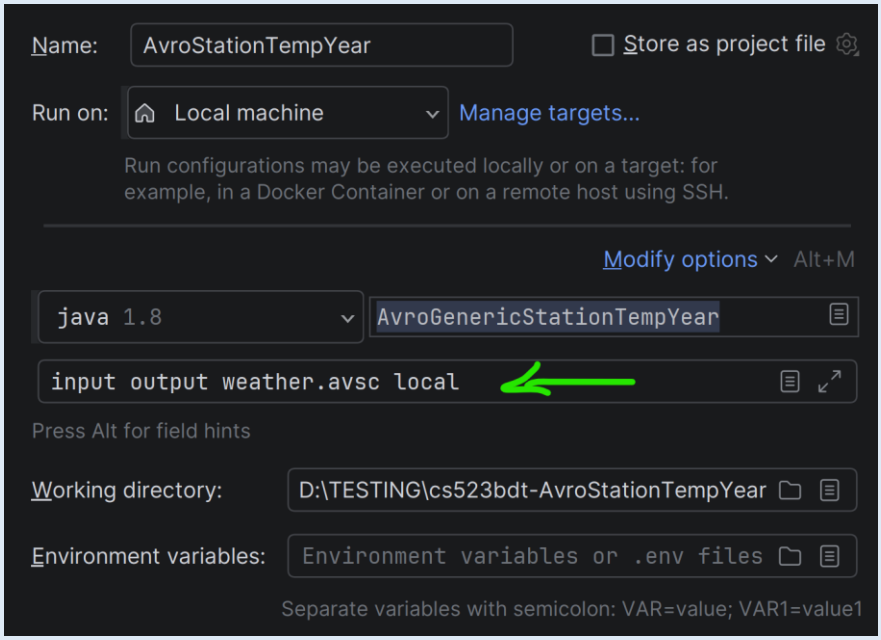
1. Get the AvroStationTempYear program by cloning the following git repository:

   git clone https://github.com/himrudula/cs523bdt-AvroStationTempYear.git

   Open the project folder "cs523bdt-AvroStationTempYear" in any IDE like IntelliJ.
2. In the project structure, you should find two java files: *AvroGenericStationTempYear.java* and *NcdcLineReaderUtils.java.*
3. Note that some required lines in the *AvroGenericStationTempYear.java* are purposefully omitted or commented. You need to find those and correct the given java program to proceed to the next steps. Take help from the lecture videos if you are unsure.
4. Also, complete the given *weather.avsc* file so that the schema matches the output.
5. A small weather dataset is also given to you. Use that as your input data set.

© 2026-28 Mrudula Mukadam, Maharishi International University * all rights reserved

6. You need to add the runtime arguments as shown below:



7. Run the program and check the output avro data file using avro-tools. Follow the steps for this from earlier section of running Avro Word Count.

8. Now add the "order" parameter to the schema file so that the final output will have stationId in ascending but the temperature in descending order as shown in the sample output below:

011990-99999	**10.0**	**1950**
011990-99999	**8.6**	**1901**
011990-99999	**7.9**	**1930**
……	**…**	**…**
012650-99999	**12.0**	**1960**
012650-99999	**10.5**	**2015**
……		

Submit the new schema and the newly generated avro file along with a screenshot of the json output using avro-tools.

No need to run this in the pseudo-distributed mode.

© 2026-28 Mrudula Mukadam, Maharishi International University * all rights reserved

3. Avro Max Temperature per Year with Station-Id

- Use the same NCDC small dataset for this problem.
- **Now this time the output file should show the maximum temperature per year along with the station that reported it.**
- The format should be as follows: Latest year is shown first.

Year	MaxTemp	StationId
2018	10.0	029720-99999
1950	12.8	227070-99999
1910	7.2	029720-99999

……

For simplicity, if the same MaxTemp is reported by multiple stations then you can just show any one of the station ids. Alternatively, you can show a list of all those station ids.

Submit the java files, schema file and the newly generated avro output file along with a screenshot of the json output using avro-tools.

No need to run this in the pseudo-distributed mode.

© 2026-28 Mrudula Mukadam, Maharishi International University * all rights reserved